Space Shuttle Main Engine

November 11, 2026

Contents

1 Introduction **1**

2 Oxygen **2**
2.1 First Enthalpy Exchanger 2
2.2 Combustion Chamber 3
2.3 Chamber Temperature (Second Enthalpy Exchanger) 7
2.4 Nozzle Performance 7

3 Fluorine **8**
3.1 First Enthalpy Exchanger 9
3.2 Combustion Chamber 9
3.3 Results 9
3.4 Data 10

A Code listings **10**
A.1 SSMEMain.py 10
A.2 exchanger.py 12
A.3 Iterater.py 23

1 Introduction

The Space Shuttle Main Engine (SSME) is a liquid-fueled rocket engine used to power the Space Shuttle Orbiter. This report will discuss the design and methodologies of the SSME code, as well as the results of the code. For this project, we were given the parameters below:

Combustion Chamber Data	
Fuel	Hydrogen
Oxidizer	O_2
Fuel/Oxidizer Ratio	0.166
Hydrogen Injection Temperature	850 K
Oxidizer Injection Temperature	530 K
Chamber Pressure	204 atm

R_t	5.15 inch
R_e	45.35 inch
θ_p	32 degree

2 Oxygen

2.1 First Enthalpy Exchanger

We begin with computing the equivalence ratio ϕ for the oxygen first enthalpy exchanger. The equivalence ratio is defined as the ratio of the mass of fuel to the mass of oxidizer. The mass of fuel is the mass of hydrogen, and the mass of oxidizer is the mass of oxygen. The mass of hydrogen is $\phi \frac{mol_{O_2}}{mol_{H_2}} = \Phi$.

From here we are to calculate the stndard heat of formation by:

$$\Delta\hat{h}_{H_2} = \hat{h}_{H_2}(850) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{H_2} p}{\hat{R}T^2} dp \tag{1}$$

$$\Delta\hat{h}_{O_2} = \hat{h}_{O_2}(530) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{O_2} p}{\hat{R}T^2} dp \tag{2}$$

Species	$\hat{b}$
H_2	0.0266
O_2	0.0318
H_2O	0.03049
N_2	0.0391
F_2	0.02896
HF	0.0739

Where $\hat{b}$ is read from a table developed in `exchanger.py` file. $\Delta\hat{h}_i$ is calculated from both hydrogen and oxygen. Once we calculate $\Delta\hat{h}_i$ for each, we can calculate the enthalpy of the mixture.

$$\Delta\hat{h}_{mix} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{3}$$

With $\Phi = 2\phi$ moles of H_2 per mole of O_2, complete combustion (no dissociation) gives

$$O_2 + \Phi H_2 \rightarrow 2H_2O + (\Phi - 2)H_2 \tag{4}$$

The energy available to heat the products is the propellant enthalpy plus the heat released forming two moles of water:

$$\Delta\hat{h}_{total} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} - 2\Delta\hat{h}^{298}_{f,H_2O} \tag{5}$$

The first estimate of the chamber temperature, T_{FEE}, is the single temperature at which the products absorb exactly that energy. It is found with `scipy.optimize.brentq`:

$$2\left[\hat{h}_{H_2O}(T_{FEE}) - \hat{h}_{H_2O}(298)\right] + (\Phi - 2)\left[\hat{h}_{H_2}(T_{FEE}) - \hat{h}_{H_2}(298)\right] = \Delta\hat{h}_{total} \tag{6}$$

This gives $T_{FEE} \approx 4617$ K. Because it ignores dissociation it is an upper bound; the chamber temperature comes from the second enthalpy exchanger in Section 2.3.

2.2 Combustion Chamber

For the Combustion Chamber, we have the two equations below:

$$H_2O \rightarrow OH + H \tag{7}$$

$$H_2 \rightarrow 2H \tag{8}$$

The Gibbs Free Energy for each species is given by:

$$\hat{g}_{H_2O} = \texttt{h2o_table_g}(T_C) \tag{9}$$

$$\hat{g}_{H_2} = \texttt{h2_table_g}(T_C) \tag{10}$$

$$\hat{g}_{OH} = \texttt{oh_table_g}(T_C) \tag{11}$$

$$\hat{g}_{H} = \texttt{h_table_g}(T_C) \tag{12}$$

We can then calculate the Gibbs Free Energy of the reaction by:

$$\Delta\hat{g}_{H_2} = 2\hat{g}_H - \hat{g}_{H_2} \tag{13}$$

$$\Delta\hat{g}_{H_2O} = \hat{g}_H + \hat{g}_{OH} - \hat{g}_{H_2O} \tag{14}$$

Now is the point where we involve the **Law of Mass Action**:

Species	$\nu'_{(i)}$	$\nu''_{(i)}$	$\nu''_{(i)} - \nu'_{(i)}$
H_2O	1	0	-1
OH	0	1	1
H	0	1	1

We an now use:

$$K_n = \prod_{i=1}^{N_s} X_{(n)}^{(\nu''_{(i)} - \nu'_{(i)})} = (\frac{p}{p_a})^{-\sigma_v} K_p \tag{15}$$

Where;

$$\sigma_v = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \tag{16}$$

$$K_p = e^{-\frac{\Delta G}{\hat{R}T}} \tag{17}$$

$$\Delta G = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \hat{g}_{(i)} \tag{18}$$

Both reactions have $\sigma_v = +1$, so we can solve for the equilibrium constant K_n.

$$K_{H_2O} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta \hat{g}_{H_2O}}{\hat{R}T_C}} \tag{19}$$

$$K_{H_2} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta \hat{g}_{H_2}}{\hat{R}T_C}} \tag{20}$$

Now we move into the chemistry problem.
Atom Balance:

$$H_2 + lO_2 \rightarrow N_{H_2O}H_2O + N_{H_2}H_2 + N_H H + N_{OH}OH \tag{21}$$

Recall, due to the fule being Hydrogen;

$$mass_{fuel} = 2.02 \tag{22}$$

$$mass_{Oxidizer} = \frac{mass_{fuel}}{\phi} \tag{23}$$

$$l = \frac{mass_{Oxidizer}}{2\hat{m}_{O_2}} \tag{24}$$

Where $\hat{m}_{O_2}$ is the molar mass of Oxygen. And ϕ is the Fuel to Oxidizer ratio, and not the equivalence ratio. Revisiting the **Atom Balance** equation, we can now solve for the number of moles of each species.

The atom balances for hydrogen and oxygen are:

$$H: \quad 2 = 2N_{H_2O} + 2N_{H_2} + N_H + N_{OH} \tag{25}$$

$$O: \quad 2l = N_{H_2O} + N_{OH} \tag{26}$$

That is four unknowns with only two equations, so the Law of Mass Action supplies the other two. The mole fractions are normalized by the full total,

$$X_{(i)} = \frac{N_{(i)}}{N_{Total}}, \qquad N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}, \tag{27}$$

so the two equilibrium relations become

$$K_{H_2O} = \frac{X_H X_{OH}}{X_{H_2O}} = \frac{N_H N_{OH}}{N_{H_2O} N_{Total}} \tag{28}$$

$$K_{H_2} = \frac{X_H^2}{X_{H_2}} = \frac{N_H^2}{N_{H_2} N_{Total}} \tag{29}$$

The four coupled equations (two atom balances and two equilibrium relations) are solved with `scipy.optimize.fsolve` in terms of $\ln N_{(i)}$, which keeps every mole number positive. The starting guess is complete combustion, $N_{H_2O} = 2l$ and $N_{H_2} = 1 - 2l$. At the chamber temperature T_c (Section 2.3) the solution is:

$$N_{H_2O} = 0.6763, \quad N_{H_2} = 0.2559, \quad N_H = 0.0530, \quad N_{OH} = 0.0827 \tag{30}$$

Species	$N_{(i)}$	$X_{(i)}$	$m_{(i)}$	$Y_{(i)}$	$\hat{c}_{p(i)}$	$c_{p(i)}$	$Y_{(i)}c_p$
H_2O	N_{H_2O}	$X_{H_2O} = \frac{N_{H_2O}}{N_{Total}}$	$\hat{m}N_{H_2O}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2O)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2O}c_p$
H_2	N_{H_2}	$X_{H_2} = \frac{N_{H_2}}{N_{Total}}$	$\hat{m}N_{H_2}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2}c_p$
H	N_H	$X_H = \frac{N_H}{N_{Total}}$	$\hat{m}N_H$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	Y_Hc_p
OH	N_{OH}	$X_{OH} = \frac{N_{OH}}{N_{Total}}$	$\hat{m}N_{OH}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(OH)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{OH}c_p$

Now we can calculate c_p, we sum the $Y_{(i)}c_p$ values for each species.

$$c_p = \sum_i Y_{(i)}c_p \tag{31}$$

$$c_p = Y_{H_2O}c_p + Y_{H_2}c_p + Y_Hc_p + Y_{OH}c_p \tag{32}$$

We perform now calulate the gas constant R.

$$R = \frac{\hat{R}}{m_{Total}} \tag{33}$$

where $m_{Total} = \sum_i X_{(i)}m_{(i)}$ is the mixture molar mass. The mole fractions are normalized by $N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}$, which exceeds one once dissociation produces H and OH.

$$c_v = c_p - R \tag{34}$$

$$\gamma = \frac{c_p}{c_p - \frac{\hat{R}}{\sum_i X_{(i)}m_{(i)}}} = \frac{c_p}{c_p - \frac{\hat{R}}{m_{Total}}} = \frac{c_p}{c_p - R} = \frac{c_p}{c_v} \tag{35}$$

2.3 Chamber Temperature (Second Enthalpy Exchanger)

Breaking H_2O and H_2 apart into H and OH absorbs energy, so the dissociated products reach a lower temperature than T_{FEE}. The composition depends on temperature through K_n, and the temperature depends on the composition through the energy balance, so the two are solved together. For a trial temperature the combustion-chamber step gives the equilibrium mixture, and the energy balance per mole of O_2 is checked:

$$\sum_i N_{(i)} \left[\Delta\hat{h}^{298}_{f(i)} + \hat{h}_{(i)}(T_c) - \hat{h}_{(i)}(298) \right] = \Phi \Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{36}$$

`brentq` adjusts the temperature between 2000 K and T_{FEE} until both sides match. The sensible enthalpies of H_2, O_2 and H_2O come straight from the tables, H uses its constant c_p, and OH is built from its tabulated enthalpy of formation:

$$\hat{h}_{OH}(T) - \hat{h}_{OH}(298) = \Delta\hat{h}_{f,OH}(T) - \Delta\hat{h}_{f,OH}(298) + \frac{1}{2}\left[\hat{h}_{H_2}(T) - \hat{h}_{H_2}(298)\right] + \frac{1}{2}\left[\hat{h}_{O_2}(T) - \hat{h}_{O_2}(298)\right] \tag{37}$$

The same construction applied to H_2O reproduces its tabulated sensible enthalpy within 0.07%. The result is $T_c \approx 3949$ K, which is used for γ, R and the nozzle. A Cantera equilibrium calculation with the same inputs gives 3881 K; most of the difference is that this model leaves out O and O_2.

2.4 Nozzle Performance

We begin by defining the Area Ratio:

$$A_r = \frac{\pi R_e^2}{\pi R_t^2} \tag{38}$$

Allowing for us to solve for the Mach number:

$$A_r = \left(\frac{\gamma+1}{2}\right)^{-\frac{\gamma+1}{2(\gamma-1)}} \frac{1}{M_e}\left[1 + \frac{\gamma-1}{2}M_e^2\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{39}$$

We can solve for Mach number M_e by making use of the `scipy.optimize.fsolve` function, which returns the roots of a non-linear function. After solving for M_e we can calculate the exit velocity v_e, exit Pressure p_e and exit Temperature T_e.

$$T_e = T_c \left[1 + \frac{\gamma - 1}{2} M_e^2\right]^{-1} \tag{40}$$

$$p_e = p_c \left[1 + \frac{\gamma - 1}{2} M_e^2\right]^{-\frac{\gamma}{\gamma - 1}} \tag{41}$$

$$v_e = M_e \sqrt{\gamma R T_e} \tag{42}$$

After solving for (Eq. 42), (Eq. 41) and (Eq. 40) we can calculate the mass flow rate $\dot{m}$.

$$\dot{m} = \frac{A_t p_c \sqrt{\gamma}}{\sqrt{R T_c}} \left[\frac{2}{\gamma + 1}\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{43}$$

Then once you have the mass flow rate you can calculate the thrust $\mathcal{T}$, specific impulse I_{sp} and the thrust coefficient C_T.

$$\mathcal{T} = \dot{m} v_e + (p_e - p_a) A_e \tag{44}$$

where p_a is the ambient pressure ($p_a = 0$ in vacuum, $p_a = 101325$ Pa at sea level).

$$I_{sp} = \frac{\mathcal{T}}{\dot{m} g_0} \tag{45}$$

$$C_T = \frac{\mathcal{T}}{p_c A_t} \tag{46}$$

3 Fluorine

The same engine is run with fluorine in place of oxygen, at the same equivalence ratio, injection temperatures (H_2 at 850 K, F_2 at 530 K), chamber pressure and nozzle. The stoichiometric reaction is $H_2 + F_2 \rightarrow 2HF$, so $\phi = 1.317$ moles of H_2 per mole of F_2, an oxidizer/fuel mass ratio of 14.3.

3.1 First Enthalpy Exchanger

Complete combustion, per mole of F_2, is

$$F_2 + \phi H_2 \rightarrow 2HF + (\phi - 1)H_2 \tag{47}$$

and T_{FEE} follows from

$$2\left[\hat{h}_{HF}(T_{FEE}) - \hat{h}_{HF}(298)\right] + (\phi - 1)\left[\hat{h}_{H_2}(T_{FEE}) - \hat{h}_{H_2}(298)\right] = \phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{F_2} - 2\Delta\hat{h}^{298}_{f,HF} \tag{48}$$

which gives $T_{FEE} \approx 7107$ K.

3.2 Combustion Chamber

Both H_2 and HF dissociate at these temperatures, so the products are HF, H_2, H and F:

$$F: \quad 2 = N_{HF} + N_F, \qquad H: \quad 2\phi = N_{HF} + 2N_{H_2} + N_H \tag{49}$$

$$K_{H_2 \rightarrow 2H} = \frac{N_H^2}{N_{H_2} N_{Total}}, \qquad K_{HF \rightarrow H+F} = \frac{N_H N_F}{N_{HF} N_{Total}} \tag{50}$$

with $K_n = (p/p^*)^{-1} e^{-\Delta\hat{g}/\hat{R}T}$ for both reactions. F_2 is left out of the products; at the chamber conditions $X_{F_2} \approx 10^{-5}$. The system is solved in $\ln N_{(i)}$ as for oxygen, and the chamber temperature again comes from the energy balance on the equilibrium products.

3.3 Results

	H_2/O_2	H_2/F_2
T_{FEE}	4617 K	7107 K
T_c	3949 K	5190 K
Molar mass	13.26 g/mol	15.43 g/mol
γ (frozen)	1.199	1.319
$\dot{m}$	457.9 kg/s	445.7 kg/s
I_{sp}, vacuum	478.8 s	459.5 s
I_{sp}, sea level	384.8 s	362.8 s

The equilibrium mole fractions are $X_{HF} = 0.678$, $X_{H_2} = 0.082$, $X_H = 0.159$ and $X_F = 0.082$. About a quarter of the gas is atoms. Because the

nozzle model keeps the chamber composition frozen, the energy stored in those atoms is never released by recombination, and the fluorine I_{sp} comes out below the oxygen one even though the chamber is 1240 K hotter. A shifting-equilibrium nozzle would recover much of that energy, so this result is a lower bound.

3.4 Data

The course tables have no data for atomic fluorine and no Gibbs energy for HF, and the H table stopped at 5000 K. These were taken from the NIST-JANAF thermochemical tables: the F atom table, the $\Delta\hat{g}_f$ column for HF, and the 5100–6000 K rows for H. A calculation that uses JANAF for every species gives $T_c = 5197.5$ K and the same mole fractions to within 0.001. Six $H^* - H^*_{298}$ entries in the course tables were typos that broke the slope set by c_p and were corrected: H_2 at 1900 and 2400 K, F_2 at 1700 and 4500 K, and HF at 3400 and 3700 K.

A Code listings

A.1 SSMEMain.py

```
import pandas as pd
from exchanger import *


if __name__ == '__main__':

    p_starting = 204 #starting pressure in atm
    p_starting = p_starting * 101325 #convert to pascals
    FuelOxidizer_ratio = 0.166
    # equivalence ratio: (mol H2 per mol O2) / 2,
   stoichiometric being 2 H2 + O2
    phi = (FuelOxidizer_ratio * MOLAR_MASS['O2'] / MOLAR_MASS
   ['H2'])/2
    Hydr_inject_temp = 850 #K
    Ox_inject_temp = 530   #K
    ThroatRadius = 5.15 #inches
    ExitRadius = 45.35 #inches
    FEE_T, FEE_del_h_total, del_h2_hat, b_hat_table,
   h_reactants = FEE(Hydr_inject_temp, p_starting,
   Ox_inject_temp, phi)
```

```
17      print('phi = ', phi)
18
19      print('Temperature of the first enthalpy exchanger (
        complete combustion) = ', FEE_T, 'K')
20      print('Total enthalpy change in the first enthalpy
        exchanger = ', FEE_del_h_total, 'J')
21
22      # second enthalpy exchanger: equilibrium (dissociated)
        products carry the reactant enthalpy
23      T_c = ChamberTemperature(FEE_T, p_starting,
        FuelOxidizer_ratio, h_reactants)
24      print('Chamber temperature (equilibrium, with
        dissociation) = ', T_c, 'K')
25
26      gamma, R, cp_total, cv, LawOfMassAction_df = OCC(T_c,
        p_starting, FuelOxidizer_ratio)
27      print('gamma = ', gamma)
28
29      p_sea_level = 101325 # Pa
30      m_dotOx, v_eOx, p_eOx, ThrustOx, c_TOx, I_spOx, T_e_Ox,
        M_eOx = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, T_c,
         R, p_starting)
31      _, _, _, ThrustOx_SL, c_TOx_SL, I_spOx_SL, _, _ =
        OxNozzlePerf(ThroatRadius, ExitRadius, gamma, T_c, R,
        p_starting, p_sea_level)
32
33      print('Ox exit Mach number = ', M_eOx)
34      print('Ox mass flow rate = ', m_dotOx, 'kg/s')
35      print('Ox exit velocity = ', v_eOx, 'm/s')
36      print('Ox exit pressure = ', p_eOx, 'Pa')
37      print('Ox exit temperature = ', T_e_Ox, 'K')
38      print('Ox thrust (vacuum) = ', ThrustOx, 'N')
39      print('Ox thrust coefficient (vacuum) = ', c_TOx)
40      print('Ox specific impulse (vacuum) = ', I_spOx, 's')
41      print('Ox thrust (sea level) = ', ThrustOx_SL, 'N')
42      print('Ox thrust coefficient (sea level) = ', c_TOx_SL)
43      print('Ox specific impulse (sea level) = ', I_spOx_SL, 's
        ')
44
45
46      print('Fluorine Problem')
47      # Same injection temperatures and equivalence ratio, with
         F2 in place of O2:
48      # stoichiometric H2 + F2 -> 2HF, so phi is also the mol
        H2 per mol F2
```

```
    F2_inject_temp = Ox_inject_temp
    FEE_T_F, del_h_total_F, h_reactants_F = FEE_F(
Hydr_inject_temp, p_starting, F2_inject_temp, phi)
    print('Oxidizer/fuel mass ratio (Fluorine) = ',
MOLAR_MASS['F2']/(phi*MOLAR_MASS['H2']))
    print('Temperature of the first enthalpy exchanger (
Fluorine, complete combustion) = ', FEE_T_F, 'K')

    T_c_F = FluorineChamberTemperature(FEE_T_F, p_starting,
phi, h_reactants_F)
    print('Chamber temperature (Fluorine, equilibrium, with
dissociation) = ', T_c_F, 'K')

    gamma_F, R_F, cp_total_F, cv_F, LawOfMassAction_df_F =
FCC(T_c_F, p_starting, phi)
    print('gamma (Fluorine) = ', gamma_F)

    m_dotF, v_eF, p_eF, ThrustF, c_TF, I_spF, T_e_F, M_eF =
OxNozzlePerf(ThroatRadius, ExitRadius, gamma_F, T_c_F, R_F
, p_starting)
    _, _, _, ThrustF_SL, c_TF_SL, I_spF_SL, _, _ =
OxNozzlePerf(ThroatRadius, ExitRadius, gamma_F, T_c_F, R_F
, p_starting, p_sea_level)

    print('F exit Mach number = ', M_eF)
    print('F mass flow rate = ', m_dotF, 'kg/s')
    print('F exit velocity = ', v_eF, 'm/s')
    print('F exit pressure = ', p_eF, 'Pa')
    print('F exit temperature = ', T_e_F, 'K')
    print('F thrust (vacuum) = ', ThrustF, 'N')
    print('F thrust coefficient (vacuum) = ', c_TF)
    print('F specific impulse (vacuum) = ', I_spF, 's')
    print('F thrust (sea level) = ', ThrustF_SL, 'N')
    print('F thrust coefficient (sea level) = ', c_TF_SL)
    print('F specific impulse (sea level) = ', I_spF_SL, 's')
```

A.2 exchanger.py

```
from Iterater import *
from scipy.optimize import fsolve
import pandas as pd
import numpy as np
from scipy.optimize import brentq

# Molar masses in g/mol, used everywhere (SSMEMain, FEE, OCC)
```

```
MOLAR_MASS = {'H2': 2.01588, 'O2': 31.9988, 'H2O': 18.01528, 'OH': 17.00734, 'H': 1.00794,
              'F2': 37.99681, 'HF': 20.00634, 'F': 18.99840}
R_HAT = 8.314  # J/mol-K


def FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi):
    '''
    This is a function that returns the calculations of the first enthalpy exchanger
    :param Hydr_inject_temp: temperature of hydrogen injection in K
    :param p_starting: starting pressure in Pa
    :param Ox_inject_temp: temperature of oxygen injection in K
    :param phi: equivalence ratio
    :return T the complete-combustion (no dissociation) temperature in K: the first estimate
    :return del_h_total enthalpy available to heat the products, J per mol O2
    :return del_h2_hat the enthalpy of the injected H2, J/mol
    :return b_hat_table the table of b_hat values
    :return h_reactants enthalpy of the injected propellants, J per mol O2
    '''


    # Van der Waals co-volume b_hat (L/mol)
    b_hat_table = pd.DataFrame({'substance': ['H2', 'H2O', 'O2', 'N2'],
                                'b_hat': [0.0266, 0.03049, 0.0318, 0.0391]})

    b_hat_h2 = b_hat_table['b_hat'][0]
    b_hat_o2 = b_hat_table['b_hat'][2]

    # injected propellant enthalpy (J/mol): sensible + real-gas pressure correction b*(p - p_ref)
    del_h2_hat = h2_tableH(Hydr_inject_temp) + b_hat_h2*(p_starting - 101325)/1000
    del_o2_hat = o2_tableH(Ox_inject_temp) + b_hat_o2*(p_starting - 101325)/1000
    mols_h2 = phi*2
    mols_o2 = 1

```

```
    # Complete combustion, per mole of O2: O2 + 2phi H2 -> 2 H2O + (2phi - 2) H2
    # Energy available to heat the products = reactant enthalpy + heat released forming 2 H2O
    del_h_total = del_h2_hat*mols_h2 + del_o2_hat*mols_o2 - 2*H_F_298['H2O']

    # First estimate of the chamber temperature: the single T at which the products'
    # sensible enthalpy equals that energy (no dissociation yet, so this is an upper bound)
    def energy_residual(T):
        return 2*sensible_enthalpy('H2O', T) + (mols_h2 - 2)*sensible_enthalpy('H2', T) - del_h_total

    T = brentq(energy_residual, 300, 4900)
    h_reactants = del_h2_hat*mols_h2 + del_o2_hat*mols_o2

    return T, del_h_total, del_h2_hat, b_hat_table, h_reactants


def EquilibriumTemperature(T_upper, h_reactants, products):
    '''
    Second enthalpy exchanger: the temperature at which the equilibrium products carry
    the same enthalpy as the injected propellants. Dissociation absorbs energy, so the
    answer lies below the complete-combustion estimate T_upper.
    :param T_upper: complete-combustion temperature in K (upper bracket)
    :param h_reactants: enthalpy of the injected propellants, J per mol oxidizer
    :param products: function T -> {species: mol per mol oxidizer} at equilibrium
    :return: temperature in K
    '''
    def energy_residual(T):
        return sum(n*total_enthalpy(sp, T) for sp, n in products(T).items()) - h_reactants

    return brentq(energy_residual, 2000, T_upper, xtol=1e-6)


```

```
72 def ChamberTemperature(FEE_T, p_starting, FO_ratio,
       h_reactants):
73     '''
74     Adiabatic chamber temperature of the H2/O2 chamber, with
       dissociation
75     :param FEE_T: complete-combustion temperature from FEE in
       K (upper bracket)
76     :param p_starting: chamber pressure in Pa
77     :param FO_ratio: fuel to oxidizer mass ratio
78     :param h_reactants: enthalpy of the injected propellants,
       J per mol O2
79     :return T_c: chamber temperature in K
80     '''
81     l = MOLAR_MASS['H2']/(FO_ratio*MOLAR_MASS['O2'])  # mol
       O2 per mol H2, as in OCC
82     species = ['H2O', 'H2', 'H', 'OH']                  #
       order of OCC's mole table
83
84     def products(T):
85         mols = OCC(T, p_starting, FO_ratio)[4]['mols'][0:4]
86         # OCC works per mol H2; dividing by l puts the
       products per mol O2
87         return {sp: float(n)/l for sp, n in zip(species, mols
       )}
88
89     return EquilibriumTemperature(FEE_T, h_reactants,
       products)
90
91 def OCC(FEE_T, p_starting, FO_ratio):
92     '''
93     This is a function that returns the calculations of the
       oxygen combustion chamber
94     :param FEE_T: temperature of the first enthalpy exchanger
       in K
95     :param p_starting: starting pressure in Pa
96     :param FO_ratio: fuel to oxidizer ratio
97     :return gamma: the ratio of specific heats
98     :return R: the gas constant
99     :return cp_total: the total specific heat capacity
100    :return cv : the specific heat capacity at constant
       volume
101    :return LawOfMassAction_df: the dataframe of the law of
       mass action
102    '''
103
```

```
    # The chemical equation is H2O -> H + OH
    molfrac_h2o_react = 1
    molfrac_h_react = 0
    molfrac_oh_react = 0
    molfrac_h2o_prod = 0
    molfrac_h_prod = 1
    molfrac_oh_prod = 1

    # The chemical equation is H2 -> 2H
    molfrac_h2_react = 1
    molfrac_2h_react = 0
    molfrac_2h_prod = 2
    molfrac_h2_prod = 0

    molfrac_h2o_total = molfrac_h2o_prod - molfrac_h2o_react
    molfrac_h_total = molfrac_h_prod - molfrac_h_react
    molfrac_oh_total = molfrac_oh_prod - molfrac_oh_react
    sigma_nu = molfrac_h2o_total + molfrac_h_total +
molfrac_oh_total
    # print('sigma_nu = ', sigma_nu)

    molfrac_h2_total = molfrac_h2_prod - molfrac_h2_react
    molfrac_2h_total = molfrac_2h_prod - molfrac_2h_react
    sigma_nu = molfrac_h2_total + molfrac_2h_total
    # print('sigma_nu = ', sigma_nu)

    # Gibbs free energy of formation
    g_f_h2o = h2o_table_g(FEE_T)
    g_f_h = h_table_g(FEE_T)
    g_f_oh = oh_table_g(FEE_T)
    g_f_h2 = h2_table_g(FEE_T)

    # Gibbs free energy of reaction: dG = sum_i (nu''_(i) -
nu'_(i)) g_f(i), i.e. products minus reactants
    g_r_h2 = 2*g_f_h - g_f_h2             # H2  -> 2H
    g_r_h2o = g_f_h + g_f_oh - g_f_h2o    # H2O -> H + OH

    k_h2 = ((p_starting/101325)**-1)*np.exp(-g_r_h2/(R_HAT*
FEE_T))
    k_h2o = ((p_starting/101325)**-1)*np.exp(-g_r_h2o/(R_HAT*
FEE_T))


    # k_h2o = 0.007149
    # k_h2 = 0.011874839
```

```
145
146     # mol O2 per mol H2 fed: (1/FO_ratio) kg O2 per kg H2,
    converted to moles
147     l = MOLAR_MASS['H2']/(FO_ratio*MOLAR_MASS['O2'])
148
149     # Products per mole of H2 fed: H2 + l O2 -> N_H2O H2O +
    N_H2 H2 + N_H H + N_OH OH
150     #   H atoms:  2N_H2O + 2N_H2 + N_H + N_OH = 2
151     #   O atoms:   N_H2O + N_OH               = 2l
152     #   H2O -> H + OH:  X_H X_OH / X_H2O = N_H N_OH / (N_H2O
    N_total) = k_h2o
153     #   H2  -> 2H:      X_H^2 / X_H2     = N_H^2   / (N_H2
    N_total)   = k_h2
154     # Solved in log(N) so every mole number stays positive (
    no root picking needed).
155     def residuals(log_n):
156         n_h2o, n_h2, n_h, n_oh = np.exp(log_n)
157         n_tot = n_h2o + n_h2 + n_h + n_oh
158         return [2*n_h2o + 2*n_h2 + n_h + n_oh - 2,
159                 n_h2o + n_oh - 2*l,
160                 np.log(n_h*n_oh/(n_h2o*n_tot)) - np.log(k_h2o
    ),
161                 np.log(n_h**2/(n_h2*n_tot)) - np.log(k_h2)]
162
163     # start from complete combustion with a little
    dissociation
164     guess = np.log([2*l, 1 - 2*l, 0.01, 0.01])
165     log_n, info, ier, msg = fsolve(residuals, guess,
    full_output=True, xtol=1e-12)
166     if ier != 1 or max(abs(r) for r in residuals(log_n)) > 1e
    -9:
167         raise RuntimeError(f'OCC equilibrium solve did not
    converge: {msg}')
168
169     N_h2o, N_h2, N_h, N_oh = (float(n) for n in np.exp(log_n)
    )
170     N_total = N_h + N_oh + N_h2 + N_h2o
171
172     molar_mass_h2o = MOLAR_MASS['H2O']
173     molar_mass_h = MOLAR_MASS['H']
174     molar_mass_oh = MOLAR_MASS['OH']
175     molar_mass_h2 = MOLAR_MASS['H2']
176
177     LawOfMassAction_df = pd.DataFrame({'substance': ['N_H2O',
     'N_H2', 'N_H', 'N_OH', 'Total'],
```

```
                                        'mols': [N_h2o, N_h2,
N_h, N_oh, N_total]})
 # Mole fractions must be normalized by the total moles:
once H and OH form,
 # N_total = N_H2O + N_H2 + N_H + N_OH > 1, so X_(i) != N_
(i).
 LawOfMassAction_df['X_(i)'] = LawOfMassAction_df['mols']
/ N_total
 # X_(i)*m_hat_(i): the species' contribution to the
mixture molar mass (g/mol)
 LawOfMassAction_df['m_hat_(i)'] = pd.Series([
molar_mass_h2o, molar_mass_h2, molar_mass_h, molar_mass_oh
, 0.0])\
                                     * LawOfMassAction_df['
X_(i)']

 # mixture molar mass m_hat = sum_i X_(i) m_hat_(i), over
the four species
 m_hat_total = sum(LawOfMassAction_df['m_hat_(i)'][0:4])
 LawOfMassAction_df.loc[4, 'm_hat_(i)'] = m_hat_total
 # mass fractions Y_(i) = X_(i) m_hat_(i) / m_hat
 LawOfMassAction_df['Y_(i)'] = LawOfMassAction_df['m_hat_(
i)'] / m_hat_total

 h2_cp = h2_table_cp(FEE_T)
 h2o_cp = h2o_table_cp(FEE_T)
 h_cp = h_table_cp(FEE_T)
 oh_cp = oh_table_cp(FEE_T)

 cp_hats = pd.Series([h2o_cp, h2_cp, h_cp, oh_cp])
 cps = cp_hats / pd.Series([molar_mass_h2o, molar_mass_h2,
 molar_mass_h, molar_mass_oh])
 cp_df = pd.DataFrame({'cp_hat': cp_hats, 'cp': cps})
 cp_df['Y_(i)*cp_(i)'] = cp_df['cp'] * LawOfMassAction_df[
'Y_(i)'][0:4]

 cp_total = sum(cp_df['Y_(i)*cp_(i)'])

 LawOfMassAction_df = pd.concat([LawOfMassAction_df, cp_df
], axis=1)

 R = R_HAT/m_hat_total
 cv = cp_total - R

 gamma = cp_total/cv
```

```
210
211     return gamma, R, cp_total, cv, LawOfMassAction_df
212
213 def FEE_F(Hydr_inject_temp, p_starting, F2_inject_temp, phi):
214     '''
215     First enthalpy exchanger for the H2/F2 chamber (same
    equivalence ratio as the oxygen case)
216     :param Hydr_inject_temp: temperature of hydrogen
    injection in K
217     :param p_starting: chamber pressure in Pa
218     :param F2_inject_temp: temperature of fluorine injection
    in K
219     :param phi: equivalence ratio; stoichiometric is H2 + F2
    -> 2HF, so phi = mol H2 per mol F2
220     :return T: complete-combustion (no dissociation)
    temperature in K
221     :return del_h_total: enthalpy available to heat the
    products, J per mol F2
222     :return h_reactants: enthalpy of the injected propellants
    , J per mol F2
223     '''
224     b_hat_h2 = 0.0266   # Van der Waals co-volume, L/mol
225     b_hat_f2 = 0.02896
226
227     del_h2_hat = h2_tableH(Hydr_inject_temp) + b_hat_h2*(
    p_starting - 101325)/1000
228     del_f2_hat = sensible_enthalpy('F2', F2_inject_temp) +
    b_hat_f2*(p_starting - 101325)/1000
229     h_reactants = phi*del_h2_hat + del_f2_hat
230
231     # Complete combustion, per mole of F2: F2 + phi H2 -> 2
    HF + (phi - 1) H2
232     del_h_total = h_reactants - 2*H_F_298['HF']
233
234     def energy_residual(T):
235         return 2*sensible_enthalpy('HF', T) + (phi - 1)*
    sensible_enthalpy('H2', T) - del_h_total
236
237     T = brentq(energy_residual, 300, 9900)
238
239     return T, del_h_total, h_reactants
240
241
242 def FluorineChamberTemperature(FEE_T, p_starting, phi,
    h_reactants):
```

```
    '''
    Adiabatic chamber temperature of the H2/F2 chamber, with dissociation
    :param FEE_T: complete-combustion temperature from FEE_F in K
    :param p_starting: chamber pressure in Pa
    :param phi: equivalence ratio (mol H2 per mol F2)
    :param h_reactants: enthalpy of the injected propellants, J per mol F2
    :return T_c: chamber temperature in K
    '''
    species = ['HF', 'H2', 'H', 'F']   # order of FCC's mole table

    def products(T):
        mols = FCC(T, p_starting, phi)[4]['mols'][0:4]
        return {sp: float(n) for sp, n in zip(species, mols)}

    # the H and F tables stop at 6000 K
    return EquilibriumTemperature(min(FEE_T, 6000), h_reactants, products)

def OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R, p_starting, p_ambient=0.0):
    '''
    This is a function that returns the performance of the oxidizer nozzle
    :param ThroatRadius: throat radius in inches
    :param ExitRadius: exit radius in inches
    :param gamma: ratio of specific heats (frozen, from OCC)
    :param FEE_T: chamber temperature in K
    :param R: gas constant in kJ/kg-K
    :param p_starting: chamber pressure in Pa
    :param p_ambient: ambient pressure in Pa (0 = vacuum, 101325 = sea level)
    :return: m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
    '''

    Area_ratio = (ExitRadius/ThroatRadius)**2
    # change np.float64 values to a float
    gamma = float(gamma)
    FEE_T = float(FEE_T)
    R = float(R)
    R = R*1000 # convert to J/kg-K

```

```
    M_e = float(MachSolve(gamma, Area_ratio, 5.0)[0])
    T_e = FEE_T*((1+((gamma-1)/2)*M_e**2)**(-1))
    v_e = M_e * np.sqrt(gamma * R * T_e)
    p_e = p_starting * (1 + ((gamma - 1) / 2) * M_e ** 2) **
(-gamma / (gamma - 1))
    ExitRadius = ExitRadius/39.37 # convert to meters
    ThroatRadius = ThroatRadius/39.37 # convert to


    m_dot = (p_starting * np.pi * (ThroatRadius)**2) / (np.
sqrt(FEE_T)) * \
        np.sqrt(gamma/R)*((2/(gamma+1))**((gamma+1)/(2*(gamma
-1))))

    A_e = np.pi * ExitRadius**2
    A_t = np.pi * ThroatRadius**2

    # Control-volume momentum balance: F = m_dot*v_e + (p_e -
    p_a)*A_e.
    # Chamber pressure only enters through c_T's
normalization (p_c*A_t), not the thrust itself.
    Thrust = m_dot * v_e + (p_e - p_ambient) * A_e
    c_T = Thrust / (p_starting * A_t)
    I_sp = Thrust / (m_dot * 9.81)

    return m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e

def FCC(T, p_starting, phi):
    '''
    Equilibrium composition and gas properties of the H2/F2
chamber
    Products per mole of F2 fed: F2 + phi H2 -> N_HF HF +
N_H2 H2 + N_H H + N_F F
      F atoms:  N_HF + N_F              = 2
      H atoms:  N_HF + 2N_H2 + N_H      = 2 phi
      H2 -> 2H:     N_H^2     / (N_H2 N_total) = K_n
      HF -> H + F:  N_H N_F   / (N_HF N_total) = K_n
    F2 is left out of the products: it is negligible in a
fuel-rich chamber at these temperatures.
    :param T: temperature in K
    :param p_starting: chamber pressure in Pa
    :param phi: equivalence ratio (mol H2 per mol F2)
    :return gamma, R (kJ/kg-K), cp_total (kJ/kg-K), cv,
LawOfMassAction_df
    '''
```

```
    p_ratio = p_starting/101325
    g_f_h = h_table_g(T)
    g_f_h2 = h2_table_g(T)
    g_f_f = table_interp('F_table.csv', 'g_hat', T)
    g_f_hf = table_interp('HydrogenFluoride_HF_table.csv', 'g_hat', T)

    # Gibbs free energy of reaction, products minus reactants; both reactions have sigma = +1
    k_h2 = p_ratio**-1*np.exp(-(2*g_f_h - g_f_h2)/(R_HAT*T))      # H2 -> 2H
    k_hf = p_ratio**-1*np.exp(-(g_f_h + g_f_f - g_f_hf)/(R_HAT*T))  # HF -> H + F

    def residuals(log_n):
        n_hf, n_h2, n_h, n_f = np.exp(log_n)
        n_tot = n_hf + n_h2 + n_h + n_f
        return [n_hf + n_f - 2,
                n_hf + 2*n_h2 + n_h - 2*phi,
                np.log(n_h**2/(n_h2*n_tot)) - np.log(k_h2),
                np.log(n_h*n_f/(n_hf*n_tot)) - np.log(k_hf)]

    # start from complete combustion with a little dissociation
    guess = np.log([2, phi - 1, 0.01, 0.01])
    log_n, info, ier, msg = fsolve(residuals, guess, full_output=True, xtol=1e-12)
    if ier != 1 or max(abs(r) for r in residuals(log_n)) > 1e-9:
        raise RuntimeError(f'FCC equilibrium solve did not converge: {msg}')

    N_hf, N_h2, N_h, N_f = (float(n) for n in np.exp(log_n))
    N_total = N_hf + N_h2 + N_h + N_f

    molar_masses = np.array([MOLAR_MASS['HF'], MOLAR_MASS['H2'], MOLAR_MASS['H'], MOLAR_MASS['F']])
    mols = np.array([N_hf, N_h2, N_h, N_f])
    X = mols/N_total
    m_hat_total = float(np.sum(X*molar_masses))    # mixture molar mass, g/mol
    Y = X*molar_masses/m_hat_total                 # mass fractions

    cp_hats = np.array([table_interp('
```

```
    HydrogenFluoride_HF_table.csv', 'c_p', T)*4.184,    # cal
    -> J
                          h2_table_cp(T),
                          h_table_cp(T),
                          table_interp('F_table.csv', 'cp_hat',
     T)])
     cps = cp_hats/molar_masses                          # kJ/kg-K
     cp_total = float(np.sum(Y*cps))

     LawOfMassAction_df = pd.DataFrame({'substance': ['N_HF',
    'N_H2', 'N_H', 'N_F', 'Total'],
                                        'mols': list(mols) + [
    N_total],
                                        'X_(i)': list(X) +
    [1.0],
                                        'Y_(i)': list(Y) +
    [1.0],
                                        'cp': list(cps) + [
    cp_total]})

     R = R_HAT/m_hat_total
     cv = cp_total - R
     gamma = cp_total/cv

     return gamma, R, cp_total, cv, LawOfMassAction_df
```

A.3 Iterater.py

```
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import fsolve

# Thermo tables live next to this file, so the code runs from
    any working directory
TABLES = Path(__file__).resolve().parent / 'tables'


def h2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of
    formation
    '''
```

```
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_cloest = h2_table['T'][idx]
    h_hat_close = h2_table['H*-H*_298'][idx]
    if Temp > T_cloest:
        T_upper = h2_table['T'][idx+1]
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2_table['T'][idx-1]
        h_hat_lower = h2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat*4184

    return h_hat

def f2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
    T_cloest = f2_table['T'][idx]
    h_hat_close = f2_table['H*-H*_298'][idx]
    if Temp > T_cloest:
        T_upper = f2_table['T'][idx+1]
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = f2_table['T'][idx-1]
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat*4184

    return h_hat

def o2_tableH(Temp):
```

```
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    o2_table = pd.read_csv(TABLES / 'o2_table.csv')

    idx = (o2_table['T']-Temp).abs().idxmin()
    T_cloest = o2_table['T'][idx]
    h_hat_close = o2_table['H*-H*_298'][idx]
    if Temp > T_cloest:
        T_upper = o2_table['T'][idx+1]
        h_hat_upper = o2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = o2_table['T'][idx-1]
        h_hat_lower = o2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat

    return h_hat

def hf_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    hf_table = pd.read_csv(TABLES / 'HydrogenFluoride_HF_table.csv')

    idx = (hf_table['T']-Temp).abs().idxmin()
    T_cloest = hf_table['T'][idx]
    h_hat_close = hf_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = hf_table['T'][idx+1]
        h_hat_upper = hf_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = hf_table['T'][idx-1]
        h_hat_lower = hf_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
```

```
    T_lower-T_cloest)*(Temp-T_cloest)
95      h_hat = h_hat*4184
96
97      return h_hat
98
99  def h2o_table_f(Temp):
100     '''
101     :param Temp: temperature in K
102     :return h_hat: enthalpy in J/mol also known as heat of
    formation
103     '''
104     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
105
106     idx = (h2o_table['T']-Temp).abs().idxmin()
107     T_cloest = h2o_table['T'][idx]
108     h_hat_close = h2o_table['h_hat'][idx]
109     if Temp > T_cloest:
110         T_upper = h2o_table['T'][idx+1]
111         h_hat_upper = h2o_table['h_hat'][idx+1]
112         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
113     else:
114         T_lower = h2o_table['T'][idx-1]
115         h_hat_lower = h2o_table['h_hat'][idx-1]
116         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
117
118     return h_hat
119
120 def oh_table_f(Temp):
121     '''
122     :param Temp: temperature in K
123     :return h_hat: enthalpy in J/mol also known as heat of
    formation
124     '''
125     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
126
127     idx = (oh_table['T']-Temp).abs().idxmin()
128     T_cloest = oh_table['T'][idx]
129     h_hat_close = oh_table['h_hat'][idx]
130     if Temp > T_cloest:
131         T_upper = oh_table['T'][idx+1]
132         h_hat_upper = oh_table['h_hat'][idx+1]
133         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
```

```
    else:
        T_lower = oh_table['T'][idx-1]
        h_hat_lower = oh_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return h_hat

def h_table_f(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_closest = h_table['T'][idx]
    h_hat_close = h_table['h_hat'][idx]
    if Temp > T_closest:
        T_upper = h_table['T'][idx+1]
        h_hat_upper = h_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h_table['T'][idx-1]
        h_hat_lower = h_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return h_hat

def h2o_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of formation
    :return Temp: temperature in K
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['h_f']-h_hat).abs().idxmin()
    h_hat_cloest = h2o_table['h_f'][idx]
    T_close = h2o_table['T'][idx]
    if h_hat > h_hat_cloest:
        h_hat_upper = h2o_table['h_f'][idx+1]
```

```
174         T_upper = h2o_table['T'][idx+1]
175         T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
176     else:
177         h_hat_lower = h2o_table['h_f'][idx-1]
178         T_lower = h2o_table['T'][idx-1]
179         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)
180 
181     return T
182 
183 def h2_tableT(h_hat):
184     '''
185     :param h_hat: enthalpy in J/mol also known as heat of
    formation
186     :return Temp: temperature in K
187     '''
188     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
189     h_hat = h_hat/4184
190     idx = (h2_table['H*-H*_298']-h_hat).abs().idxmin()
191     h_hat_cloest = h2_table['H*-H*_298'][idx]
192     T_close = h2_table['T'][idx]
193     if h_hat > h_hat_cloest:
194         h_hat_upper = h2_table['H*-H*_298'][idx+1]
195         T_upper = h2_table['T'][idx+1]
196         T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
197     else:
198         h_hat_lower = h2_table['H*-H*_298'][idx-1]
199         T_lower = h2_table['T'][idx-1]
200         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)
201 
202     return T
203 
204 def hf_tableT(h_hat):
205     '''
206     :param h_hat: enthalpy in J/mol also known as heat of
    formation
207     :return Temp: temperature in K
208     '''
209     f2_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
210     h_hat = h_hat/4184
211     idx = (f2_table['H*-H*_298']-h_hat).abs().idxmin()
```

```
212     h_hat_cloest = f2_table['H*-H*_298'][idx]
213     T_close = f2_table['T'][idx]
214     if h_hat > h_hat_cloest:
215         h_hat_upper = f2_table['H*-H*_298'][idx+1]
216         T_upper = f2_table['T'][idx+1]
217         T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_cloest)*(h_hat-h_hat_cloest)
218     else:
219         h_hat_lower = f2_table['H*-H*_298'][idx-1]
220         T_lower = f2_table['T'][idx-1]
221         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_cloest)*(h_hat-h_hat_cloest)
222 
223     return T
224 
225 
226 def oh_table_g(Temp):
227     '''
228     :param Temp: temperature in K
229     :return g_hat: Gibbs free energy in J/mol
230     '''
231     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
232 
233     idx = (oh_table['T']-Temp).abs().idxmin()
234     T_cloest = oh_table['T'][idx]
235     g_hat_close = oh_table['g_hat'][idx]
236     if Temp > T_cloest:
237         T_upper = oh_table['T'][idx+1]
238         g_hat_upper = oh_table['g_hat'][idx+1]
239         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
240     else:
241         T_lower = oh_table['T'][idx-1]
242         g_hat_lower = oh_table['g_hat'][idx-1]
243         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
244 
245     return g_hat
246 
247 def h_table_g(Temp):
248     '''
249     :param Temp: temperature in K
250     :return g_hat: Gibbs free energy in J/mol
251     '''
252     h_table = pd.read_csv(TABLES / 'H_table.csv')
```

```
253
254     idx = (h_table['T']-Temp).abs().idxmin()
255     T_cloest = h_table['T'][idx]
256     g_hat_close = h_table['g_hat'][idx]
257     if Temp > T_cloest:
258         T_upper = h_table['T'][idx+1]
259         g_hat_upper = h_table['g_hat'][idx+1]
260         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
261     else:
262         T_lower = h_table['T'][idx-1]
263         g_hat_lower = h_table['g_hat'][idx-1]
264         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
265
266     return g_hat
267
268 def h2o_table_g(Temp):
269     '''
270     :param Temp: temperature in K
271     :return g_hat: Gibbs free energy in J/mol
272     '''
273     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
274
275     idx = (h2o_table['T']-Temp).abs().idxmin()
276     T_cloest = h2o_table['T'][idx]
277     g_hat_close = h2o_table['g_hat'][idx]
278     if Temp > T_cloest:
279         T_upper = h2o_table['T'][idx+1]
280         g_hat_upper = h2o_table['g_hat'][idx+1]
281         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
282     else:
283         T_lower = h2o_table['T'][idx-1]
284         g_hat_lower = h2o_table['g_hat'][idx-1]
285         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
286
287     return g_hat
288
289 def h2_table_g(Temp):
290     '''
291     :param Temp: temperature in K
292     :return g_hat: Gibbs free energy in J/mol
293     '''
```

```
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_closest = h2_table['T'][idx]
    g_hat_close = h2_table['g_hat'][idx]
    if Temp > T_closest:
        T_upper = h2_table['T'][idx+1]
        g_hat_upper = h2_table['g_hat'][idx+1]
        g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2_table['T'][idx-1]
        g_hat_lower = h2_table['g_hat'][idx-1]
        g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return g_hat

def h2o_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['T']-Temp).abs().idxmin()
    T_closest = h2o_table['T'][idx]
    cp_hat_close = h2o_table['cp_hat'][idx]
    if Temp > T_closest:
        T_upper = h2o_table['T'][idx+1]
        cp_hat_upper = h2o_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2o_table['T'][idx-1]
        cp_hat_lower = h2o_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return cp_hat

def h2_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
```

```
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_cloest = h2_table['T'][idx]
    cp_hat_close = h2_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h2_table['T'][idx+1]
        cp_hat_upper = h2_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2_table['T'][idx-1]
        cp_hat_lower = h2_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def oh_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_cloest = oh_table['T'][idx]
    cp_hat_close = oh_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = oh_table['T'][idx+1]
        cp_hat_upper = oh_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = oh_table['T'][idx-1]
        cp_hat_lower = oh_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def h_table_cp(Temp):
    '''
    :param Temp: temperature in K
```

```
    :return cp_hat: heat capacity in J/mol-K
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_cloest = h_table['T'][idx]
    cp_hat_close = h_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h_table['T'][idx+1]
        cp_hat_upper = h_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h_table['T'][idx-1]
        cp_hat_lower = h_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def MachSolve(gamma, r_A, guess):
    '''
    Solve for Mach number given gamma, r_A, and guess
    :param gamma: the gamma of the gas
    :param r_A: the ratio of the areas
    :param guess: the initial guess of the Mach number
    :return: Mach number
    '''

    # M = var('M')
    front_cont = ((gamma+1)/2)**(-(gamma+1)/(2*(gamma-1)))
    sec_cont = (gamma-1)/2
    func = lambda M: (((front_cont)*((1+sec_cont*M**2)**((gamma+1)/(2*(gamma-1)))))/M) - r_A
    sol = fsolve(func, guess)

    return sol

def h2_tableS(Temp):
    '''
    :param Temp: temperature in K
    :return S_hat: entropy in J/mol-K
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

```

```
418     idx = (h2_table['T']-Temp).abs().idxmin()
419     T_cloest = h2_table['T'][idx]
420     S_hat_close = h2_table['S'][idx]
421     if Temp > T_cloest:
422         T_upper = h2_table['T'][idx+1]
423         S_hat_upper = h2_table['S'][idx+1]
424         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
425     else:
426         T_lower = h2_table['T'][idx-1]
427         S_hat_lower = h2_table['S'][idx-1]
428         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
429 
430     return S_hat
431 
432 def hf_tableS(Temp):
433     '''
434     :param Temp: temperature in K
435     :return S_hat: entropy in J/mol-K
436     '''
437     hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
438 
439     idx = (hf_table['T']-Temp).abs().idxmin()
440     T_cloest = hf_table['T'][idx]
441     S_hat_close = hf_table['S'][idx]
442     if Temp > T_cloest:
443         T_upper = hf_table['T'][idx+1]
444         S_hat_upper = hf_table['S'][idx+1]
445         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
446     else:
447         T_lower = hf_table['T'][idx-1]
448         S_hat_lower = hf_table['S'][idx-1]
449         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
450 
451     return S_hat
452 
453 def f2_tableS(Temp):
454     '''
455     :param Temp: temperature in K
456     :return S_hat: entropy in J/mol-K
457     '''
```

```
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
    T_closest = f2_table['T'][idx]
    S_hat_close = f2_table['S'][idx]
    if Temp > T_closest:
        T_upper = f2_table['T'][idx+1]
        S_hat_upper = f2_table['S'][idx+1]
        S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = f2_table['T'][idx-1]
        S_hat_lower = f2_table['S'][idx-1]
        S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(T_lower-T_closest)*(Temp-T_closest)

    return S_hat
# Me = MachSolve(1.237115, 77.5426, 5)
# print('Me = ', Me)

def table_interp(filename, column, Temp):
    '''
    Linear interpolation of one column of a thermo table in temperature
    :param filename: table file name inside ./tables
    :param column: column to interpolate
    :param Temp: temperature in K (must lie inside the table)
    :return: interpolated value
    '''
    table = pd.read_csv(TABLES / filename)
    T_min, T_max = table['T'].min(), table['T'].max()
    if not T_min <= Temp <= T_max:
        raise ValueError(f'T = {Temp} K is outside {filename} ({T_min}-{T_max} K)')
    return float(np.interp(Temp, table['T'], table[column]))


# Enthalpy of formation at 298 K, J/mol, from the tables (HF: -64.8 kcal/mol; F: NIST-JANAF)
H_F_298 = {'H2O': -241845.0, 'H2': 0.0, 'H': 217977.0, 'OH': 38985.0, 'O2': 0.0,
           'HF': -64.8*4184, 'F': 79390.0, 'F2': 0.0}


def sensible_enthalpy(species, Temp):
```

```
    '''
    Sensible enthalpy h(T) - h(298 K) in J/mol, built from the tables:
      H2, O2 : tabulated H - H_298 (H2 in kcal/mol)
      H2O    : tabulated h_f column
      H      : constant cp (monatomic ideal gas)
      OH     : from its formation enthalpy, since dh_f(OH) = h_OH - (1/2)h_H2 - (1/2)h_O2
      HF, F2 : tabulated H - H_298 (kcal/mol)
      F      : tabulated H - H298 (J/mol, NIST-JANAF)
    '''
    if species == 'H2':
        return table_interp('Hydrogen_H2_table.csv', 'H*-H*_298', Temp)*4184
    if species == 'O2':
        return table_interp('o2_table.csv', 'H*-H*_298', Temp)
    if species == 'H2O':
        return table_interp('H2O_table.csv', 'h_f', Temp)
    if species == 'H':
        return table_interp('H_table.csv', 'cp_hat', Temp)*(Temp - 298)
    if species == 'OH':
        return (table_interp('OH_table.csv', 'h_hat', Temp) - H_F_298['OH']
                + 0.5*sensible_enthalpy('H2', Temp) + 0.5*sensible_enthalpy('O2', Temp))
    if species == 'HF':
        return table_interp('HydrogenFluoride_HF_table.csv', 'H*-H*_298', Temp)*4184
    if species == 'F2':
        return table_interp('Fluorine_F2_table.csv', 'H*-H*_298', Temp)*4184
    if species == 'F':
        return table_interp('F_table.csv', 'H-H298', Temp)
    raise ValueError(f'no enthalpy data for {species}')


def total_enthalpy(species, Temp):
    '''
    Formation + sensible enthalpy in J/mol (elements H2, O2 are zero at 298 K, the tables' reference)
    '''
    return H_F_298[species] + sensible_enthalpy(species, Temp)
```